Hướng dẫn cơ chế xử lý Exception trong Java (Phần I)
Giáo viên: Nguyễn Hùng Cường

Exception (ngoại lệ): Là những tình huống bất thường xảy ra trong quá trình thực thi một chương trình Java. Ngoại lệ xảy ra khiến cho chương trình bị dừng một cách bất thường, không theo mong muốn của người dùng. Để tránh cho chương trình bị dừng một cách đột ngột không mong muốn, khi lập trình ta cần cài đặt cơ chế xử lý ngoại lệ. Trong Java, mỗi khi chương trình có lỗi xảy ra, ta thường nói chương trình Java đã ném ra ngoại lệ.

Trong Java, để cài đặt cơ chế xử lý ngoại lệ (Exception Handling), ta cần cài đặt 3 khối lệnh như sau:
- khối try: Là khối lệnh chứa các đoạn mã nguồn mà ta cho rằng có khả năng gây ra lỗi. Khối try sẽ được chứa các câu lệnh để thử lỗi.
- khối catch: Dùng để xử lý ngoại lệ. Khối catch sẽ chứa các đoạn mã dùng để xử lý mỗi khi có một ngoại lệ tương ứng xảy ra. Việc xử lý có thể bao gồm việc in thông báo, kết thúc chương trình, giải phóng bộ nhớ, gọi đến các method khác, ném ra ngoại lệ v.v…
- khối finally: Dùng để dọn dẹp và giải phóng tài nguyên. Khối finally là khối lệnh mà sẽ luôn được thực hiện, cho dù chương trình có bị lỗi hay không. Khối finally thường được sử dụng để giải phóng tài nguyên khi chương trình bị lỗi, ví dụ như: đóng kết nối đến CSDL, đóng tập tin …

Cơ chế xử lý ngoại lệ trong Java:
Mỗi khi một dòng lệnh ở khối try ném ra ngoại lệ, khối try sẽ dừng việc thực thi tại dòng đó, và điều khiển của chương trình sẽ chuyển đến các khối catch để xử lý ngoại lệ. Chương trình sẽ tìm lần lượt từng khối catch để xem ngoại lệ được ném ra có phù hợp với khối catch đó hay không. Nếu một khối catch có thể xử lý ngoại lệ, thì chương trình sẽ thực thi khối catch đó, và bỏ qua tất cả các khối catch còn lại. Nếu tất cả các khối catch đều không xử lý được ngoại lệ, thì chương trình sẽ dừng, và máy ảo Java sẽ xử lý ngoại lệ.

THẦY CƯỜNG LẬP TRÌNH

Một số điều chú ý khi viết mã xử lý ngoại lệ:

- Khối try không thể đứng một mình, mà phía sau khối try cần phải khai báo khối catch hoặc finally (hoặc cả hai).
- Sau khối try có thể khai báo nhiều khối catch (như ví dụ bên dưới). Mỗi khối catch sẽ chịu trách nhiệm xử lý một loại ngoại lệ khác nhau.
- Nên viết khối finally, để có thể giải phóng tài nguyên nếu chương trình có lỗi hoặc không.

Minh họa:

Sử dụng một trình soạn thảo Java, như NetBeans hoặc Eclipse. Tạo mới một ứng dụng Java. Đặt tên và đường dẫn chứa ứng dụng. Sau đó ta viết mã để thử lỗi. Trong khối try, ta viết một số dòng lệnh có thể sẽ gây ra ngoại lệ. Sau đó, sau khối try, ta viết một số khối catch để xử lý từng loại ngoại lệ có thể được ném ra trong khối try.

```
public static void main(String[] args) {
    try {
        //khai bao chuoi la null va truy cap den chuoi null
        String s = "demo";
        System.out.println("Do dai chuoi la: " + s.length());

        //khai bao va khoi tao mang co 3 phan tu
        //sau do truy cap den phan tu thu 6 cua mang
        int a[] = {4, 7, 9};
        System.out.println("a[5] = " + a[2]);

        //chuyen mot string khong hop le thanh so
        String s1 = "40";
        int i = Integer.parseInt(s1);
        System.out.println("So la: " + i);
```

```
    //loi chia mot so cho 0
    int x = 4;
    int y = 2;
    int z = x/y;

    System.out.println("Ket thuc khoi try");
} catch(NullPointerException e) {
    //khoi catch nay xu ly khi truy cap den doi tuong null
    System.out.println("Loi chua khoi tao doi tuong !");
    e.printStackTrace();

} catch(ArrayIndexOutOfBoundsException e) {
    //khoi catch nay xu ly khi ta truy cap den phan tu
    //vuot qua kich thuoc mang
    System.out.println("Loi tran mang !");
    e.printStackTrace();
} catch(NumberFormatException ne) {
    //khoi catch nay xu ly khi ta dinh dang so khong hop le
    System.out.println("Loi dinh dang so !");
    ne.printStackTrace();
} catch(Exception e) {
    //khoi catch xu ly duoc moi ngoai le
    System.out.println("Co loi xay ra trong qua trinh thuc thi !");
    e.printStackTrace();
}

    finally {
        System.out.println("Ket thuc chuong trinh !");
    }
}
```

Ở trong chương trình này, ta cũng cài đặt khối finally để có thể minh họa về việc thực hiện của khối này.

Đầu tiên, ta thực thi chương trình, ta có thể thấy chương trình hoạt động tốt, không ném ra ngoại lệ.

```
run:
Do dai chuoi la: 4
a[5] = 9
So la: 40
Ket thuc khoi try
Ket thuc chuong trinh !
BUILD SUCCESSFUL (total time: 0 seconds)
```

Sau đó, ta sẽ thử từng loại ngoại lệ. Đầu tiên ta gán chuỗi s bằng null, như vậy câu lệnh tiếp theo sẽ ném ra ngoại lệ NullPointerException.

```
//khai bao chuoi la null va truy cap den chuoi null
String s = null;
System.out.println("Do dai chuoi la: " + s.length());
```

Sau đó khi chạy chương trình, ta sẽ thấy chương trình ném ra ngoại lệ NullPointerException, chương trình sẽ thực thi khối catch tương ứng với ngoại lệ này. Ta có thể thấy khối finally vẫn thực thi sau khi ngoại lệ được ném ra.

```
run:
Loi chua khoi tao doi tuong !
java.lang.NullPointerException
Ket thuc chuong trinh !
        at DemoException.main(DemoException.java:20)
BUILD SUCCESSFUL (total time: 0 seconds)
```

Sau đó, ta thử một ngoại lệ khác, ta thử truy cập đến một phần tử vượt quá chiều dài mảng như hình bên dưới. Đoạn mã bên dưới sẽ có lỗi, vì mảng chỉ có kích thước là 3, nhưng ta đã truy cập đến phần tử ở vị trí thứ 6 (chỉ số là 5).

```
//khai bao chuoi la null va truy cap den chuoi null
String s = "demo";
System.out.println("Do dai chuoi la: " + s.length());

//khai bao va khoi tao mang co 3 phan tu
//sau do truy cap den phan tu thu 6 cua mang
int a[] = {4, 7, 9};
System.out.println("a[5] = " + a[5]);
```

Sau khi thực thi, ta có thể thấy chương trình đã ném ra một ngoại lệ khác thuộc kiểu ArrayIndexOutOfBoundsException, ngoại lệ này được ném ra khi chương trình đã truy cập vượt quá chiều dài mảng (lỗi tràn mảng).

```
run:
Do dai chuoi la: 4
Loi tran mang !
java.lang.ArrayIndexOutOfBoundsException: 5
Ket thuc chuong trinh !
        at DemoException.main(DemoException.java:25)
BUILD SUCCESSFUL (total time: 0 seconds)
```

Tiếp theo, ta sửa lại lỗi này, và ta sửa lại mã nguồn, để thực hiện xử lý một lỗi khác. Ta sửa lại chuỗi ở trên thành “abc”, rồi chuyển đổi chuỗi này thành số nguyên (chắc chắn sẽ ném ra ngoại lệ, vì chuỗi “abc” không thể chuyển được thành số).

THẦY CỬU

```
//khai bao va khoi tao mang co 3 phan tu
//sau do truy cap den phan tu thu 6 cua mang
int a[] = {4, 7, 9};
System.out.println("a[5] = " + a[2]);

//chuyen mot string khong hop le thanh so
String s1 = "abc";
int i = Integer.parseInt(s1);
System.out.println("So la: " + i);
```

Khi thực thi chương trình, ta có thể thấy một ngoại lệ thuộc kiểu NumberFormatException đã được ném ra. Đây là ngoại lệ được ném ra khi ta định dạng dữ liệu sai.

```
a[5] = 9
java.lang.NumberFormatException: For input string: "abc"
Loi dinh dang so !
Ket thuc chuong trinh !
        at java.lang.NumberFormatException.forInputString(NumberFormatException.jav
        at java.lang.Integer.parseInt(Integer.java:580)
        at java.lang.Integer.parseInt(Integer.java:615)
        at DemoException.main(DemoException.java:29)
```

Tiếp theo, ta thử một ngoại lệ khác, ta khai báo hai biến x, y, rồi tiến hành chia 2 biến này, trong đó y = 0. Ở đây ta đã thực hiện phép chia cho 0, dòng lệnh này sẽ phát sinh ngoại lệ.

```
//chuyen mot string khong hop le thanh so
String s1 = "40";
int i = Integer.parseInt(s1);
System.out.println("So la: " + i);

//loi chia mot so cho 0
int x = 4;
int y = 0;
int z = x/y;

System.out.println("Ket thuc khoi try");
```

Sau khi thực thi chương trình, ta thấy ngay: Dòng lệnh trên đã hiển thị thông báo lỗi như hình bên dưới.

```
Do dai chuoi la: 4
a[5] = 9
java.lang.ArithmeticException: / by zero
So la: 40
```

THẦY CỰC LẬP TRÌNH